CS584 – Assignment 4 (Transformers)

Due by: 4/6/2026

General Instructions

Objectives

By the end of this assignment, you will:

- understand how Transformer architectures are applied to text classification
- implement Transformer-based models using both built-in and custom components
- analyze the role of attention, positional encoding, and residual connections
- compare abstraction (built-in models) vs. manual implementation
- evaluate model performance and conduct parameter studies
- You will use the IMDB sentiment classification dataset.
- You must evaluate model performance and study the effect of varying parameters.
- The grade will depend on both correctness of your implementations and thoroughness of evaluation.
- Your code must be modular, well documented, and written in Python.

Submission Instructions

- Upload your submission to Canvas.
- Submit a fully executed Python notebook with no gaps in execution. To receive full credit, please ensure: The notebook includes all cell outputs and contains no error messages. Each problem is clearly labeled in Markdown or comments (e.g., "Problem 2"). Re-run your notebook before submitting to ensure that cells are numbered sequentially starting at `[1]`.

Experiment Instructions

- Use the IMDB dataset for all experiments.
- Evaluate effects of varying hyperparameters (e.g., embedding size, number of heads, sequence length).
- Compare results between built-in and custom Transformer implementations.
- Report standard performance metrics: confusion matrix, precision, recall, F1-score, accuracy
- For parameter studies, use plots/tables with concise interpretations.

Programming Tasks

1. Dataset Preparation

1. Load the IMDB dataset using Keras: `keras.datasets.imdb.load_data()`
2. Restrict vocabulary size (e.g., 10,000–20,000 words).
3. Pad sequences to a fixed length (e.g., 200).
4. Split the dataset into: training, validation, test sets
5. Report: number of samples in each split, sequence length statistics

2. Transformer with Keras (Baseline)

In this part, you will build a Transformer-based classifier using a prebuilt Transformer encoder. The goal is to first obtain a working model using a high-level abstraction. The model must include: token embedding using `layers.Embedding`, positional embedding (you may implement using `Embedding`), one or more transformer encoder layers using: `keras_nlp.layers.TransformerEncoder`, pooling layer, dense classification head. Tasks:

1. Build the model using the prebuilt Transformer encoder.
2. Train the model.
3. Plot: training vs validation loss, training vs validation accuracy
4. Evaluate performance on the test set.

3. Custom Transformer Block Implementation

In this part, you will reimplement the Transformer encoder block manually replacing the `keras_nlp.layers.TransformerEncoder` block in step 2. The goal is to reproduce the functionality of the prebuilt encoder from Part 2 using lower-level Keras components. To achieve this you must implement `class TransformerEncoderBlock(layers.Layer)`. Your implementation must include: Multi-head self-attention, using: `tf.keras.layers.MultiHeadAttention` (apply as `attn(x, x)`); Feedforward network (Dense $\rightarrow$ activation $\rightarrow$ Dense); Residual connections (skip connections); Layer normalization(after each residual addition); Dropout (after attention and feedforward sublayers). This task focuses only on the Transformer block itself. You are not required to implement token embeddings or positional embeddings.

You may use: `tf.keras.layers.MultiHeadAttention`, `layers.Dense`, `layers.Dropout`, `layers.LayerNormalization`, `layers.Embedding`. You may **not** use the prebuilt Transformer encoders (e.g., `keras_nlp.layers.TransformerEncoder`). Your Transformer encoder block should follow:

$$\text{Input} \rightarrow \text{Multi-Head Self-Attention} \rightarrow \text{Add and Norm} \rightarrow \text{Feedforward Network} \rightarrow \text{Add Norm} \rightarrow \text{Output}$$

Implementation Tasks:

1. Implement the `TransformerEncoderBlock` class.
2. Replace the Transformer encoder in Part 2 with your custom block.
3. Train and evaluate the updated model.

4. Comparison Study

1. Compare: Keras-based Transformer (Part 2), Custom Transformer block (Part 3).
2. Report: training accuracy, validation accuracy, test accuracy, training time, number of parameters.
3. Discuss the following:
    - What functionality does the built-in implementation provide automatically?
    - Where are residual connections implemented in your custom block?
    - Why do the models perform similarly or differently?
    - Which implementation would you prefer in practice and why?

5. Parameter Study

Evaluate the effect of varying:

1. embedding dimension
2. number of attention heads
3. sequence length
4. feedforward dimension

Present results using plots or tables with concise interpretation.

Reflection

At the end of your notebook, include a 1–2 paragraph reflection:

- Key insights gained from working with Transformers
- Differences between built-in and custom implementations
- Challenges encountered during implementation
- Lessons learned about training and evaluating Transformer models

Starter Code

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Try current package first, then older one
try:
    import keras_hub
    TransformerEncoderLayer = keras_hub.layers.TransformerEncoder
except ImportError:
    import keras_nlp
    TransformerEncoderLayer = keras_nlp.layers.TransformerEncoder

# ========================
# Configuration
# ========================
max_features = 20000
maxlen = 200
embed_dim = 64
num_heads = 2
ff_dim = 64

# ========================
# Load dataset
# ========================
(x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(
    num_words=max_features
)

x_train = keras.preprocessing.sequence.pad_sequences(x_train, maxlen=maxlen)
```

```python
x_test = keras.preprocessing.sequence.pad_sequences(x_test, maxlen=maxlen)

# ========================
# Token + positional embedding
# (provided; students do not need to implement this)
# ========================
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, maxlen, vocab_size, embed_dim):
        super().__init__()
        self.token_emb = layers.Embedding(input_dim=vocab_size, output_dim=embed_dim)
        self.pos_emb = layers.Embedding(input_dim=maxlen, output_dim=embed_dim)

    def call(self, x):
        seq_len = tf.shape(x)[-1]
        positions = tf.range(start=0, limit=seq_len, delta=1)
        position_embeddings = self.pos_emb(positions)
        token_embeddings = self.token_emb(x)
        return token_embeddings + position_embeddings

# ========================
# Baseline Transformer model
# Part 2: use prebuilt Transformer encoder layer
# ========================
def build_model_builtin(num_encoder_layers=1):
    inputs = layers.Input(shape=(maxlen,))
    x = TokenAndPositionEmbedding(maxlen, max_features, embed_dim)(inputs)

    # REQUIRED: use one or more prebuilt Transformer encoder layers
    for _ in range(num_encoder_layers):
        x = TransformerEncoderLayer(
            intermediate_dim=ff_dim,
            num_heads=num_heads,
            dropout=0.1
        )(x)

    x = layers.GlobalAveragePooling1D()(x)
    x = layers.Dropout(0.1)(x)
    x = layers.Dense(20, activation="relu")(x)
    x = layers.Dropout(0.1)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)

    return keras.Model(inputs, outputs)

# ========================
# Custom Transformer encoder block
# Part 3: students implement this
# ========================
class TransformerEncoderBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()

        self.att = layers.MultiHeadAttention(
            num_heads=num_heads,
            key_dim=embed_dim
        )

        # TODO: define feedforward network
```

```
        self.ffn = None

        # TODO: define normalization layers
        self.layernorm1 = None
        self.layernorm2 = None

        # TODO: define dropout layers
        self.dropout1 = None
        self.dropout2 = None

    def call(self, inputs, training=None):
        # TODO:
        # 1. Apply self-attention: self.att(inputs, inputs)
        # 2. Apply dropout
        # 3. Add residual connection and normalize
        # 4. Apply feedforward network
        # 5. Apply dropout
        # 6. Add residual connection and normalize
        return inputs

def build_model_custom(num_encoder_layers=1):
    inputs = layers.Input(shape=(maxlen,))
    x = TokenAndPositionEmbedding(maxlen, max_features, embed_dim)(inputs)

    for _ in range(num_encoder_layers):
        x = TransformerEncoderBlock(embed_dim, num_heads, ff_dim)(x)

    x = layers.GlobalAveragePooling1D()(x)
    x = layers.Dropout(0.1)(x)
    x = layers.Dense(20, activation="relu")(x)
    x = layers.Dropout(0.1)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)

    return keras.Model(inputs, outputs)
```